C语言——面向过程编程

单元六、文件处理

第一次课 文件基本操作

一、文件基础概念

① 数据持久化的概念：

1. 内存：俗称内存调，内存中储存数据，存取速度快、一旦出现计算机断电，内存中的数据立刻消失且无法找回
2. 外存：通常包括硬盘、U盘等。外存特点：存取速度比内存慢，但是可以持久保存，不会随着断电而数据消失。在项目开发中，通常把数据永久保存到硬盘的过程称为“持久化”； 文件系统、数据库系统都属于“数据持久化”的技术手段

② 文件种类

我们可能会根据文件的扩展名给文件分类，比如图片、视频、word文档等。但是从编程角度看，文件就分为两类：文本文件与二进制文件。比如用文本编辑器（记事本）打开的字符文件（人能读懂的）成为文本文件或称字符文件。如果开是乱码，就是二进制文件，必须用专门的应用程序打开才能读取里面的二进制数据。比如图片，视频、压缩包....。文本文件英文：text

二进制文件英文：binary

③ I/O流

I/O即 input/output输入输出的简写。流（Stream）形象的比喻。即把硬盘中的数据01序列不断读取到内存中的某个区域进行处理，那么01的传递就如同水一样的流动。我们把数据从内存到硬盘或从硬盘到内存的流动过程称为I/0流。

④ buffer 缓冲区与FILE* 指针

因为普通计算机硬盘的存储空间远远大于内存，比如一个2GB的电影，不可能把数据直接从硬盘搬运到内存后再播放。通常是在内存中开辟一个存储空间，只读取某文件一部分内容，随着处理随着读取。通常把这段内存空间称为缓冲区buffer，把指向缓冲区的指针称为FILE* 即文件指针。

二、文件基本操作

① 打开文件

```
#include<stdio.h>
#include<stdlib.h>
 int main()
{
    FILE* readFile = fopen("d:\\bbb.txt","rt");//以只读方式打开文本文件：d:\bbb.txt文件。
    if (readFile)//如果文件打开成功
    {

        fclose(readFile);//关闭文件流 。
        readFile=NULL;//释放文件指针。
    }
    else
    {
```

```
        printf("文件不存在\n");
    }
```

fopen函数：以某种方式打开文件，并返回此文件在内存缓冲区的指针。fopen打开文件的方式参考下图

控制读写权限的字符串（必须指明）	
打开方式	说明
"r"	以“只读”方式打开文件。只允许读取，不允许写入。文件必须存在，否则打开失败。
"w"	以“写入”方式打开文件。如果文件不存在，那么创建一个新文件；如果文件存在，那么清空文件内容（相当于删除原文件，再创建一个新文件）。
"a"	以“追加”方式打开文件。如果文件不存在，那么创建一个新文件；如果文件存在，那么将写入的数据追加到文件的末尾（文件原有的内容保留）。
"r+"	以“读写”方式打开文件。既可以读取也可以写入，也就是随意更新文件。文件必须存在，否则打开失败。
"w+"	以“写入/更新”方式打开文件，相当于`w`和`r+`叠加的效果。既可以读取也可以写入，也就是随意更新文件。如果文件不存在，那么创建一个新文件；如果文件存在，那么清空文件内容（相当于删除原文件，再创建一个新文件）。
"a+"	以“追加/更新”方式打开文件，相当于a和r+叠加的效果。既可以读取也可以写入，也就是随意更新文件。如果文件不存在，那么创建一个新文件；如果文件存在，那么将写入的数据追加到文件的末尾（文件原有的内容保留）。
控制读写方式的字符串（可以不写）	
打开方式	说明
"t"	文本文件。如果不写，默认为`"t"`。
"b"	二进制文件。

调用 fopen() 函数时必须指明读写权限，但是可以不指明读写方式（此时默认为`"t"`）。

读写权限和读写方式可以组合使用，但是必须将读写方式放在读写权限的中间或者尾部（换句话说，不能将读写方式放在读写权限的开头）。例如：

- 将读写方式放在读写权限的末尾："rb"、"wt"、"ab"、"r+b"、"w+t"、"a+t"
- 将读写方式放在读写权限的中间："rb+"、"wt+"、"ab+"

整体来说，文件打开方式由 r、w、a、t、b、+ 六个字符拼成，各字符的含义是：

- r(read)：读
- w(write)：写
- a(append)：追加
- t(text)：文本文件

- b(binary)：二进制文件
- +：读和写

② 创建文件

```
FILE * newFile= fopen("d:\\白名单.txt","w");//w方式：  如果文件不存在，则创建一个，如果存在，则清空并覆盖新的内容
fclose(newFile);
```

③ 给现有文件改名

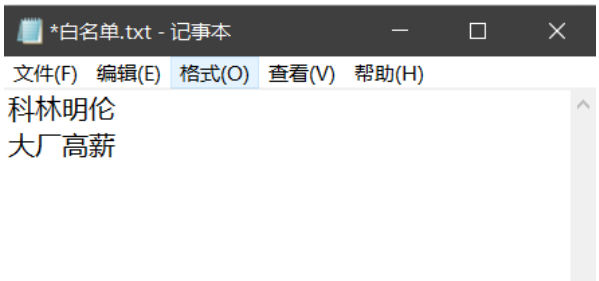
```
rename("d:\\aaa.txt","d:\\bbb.txt");//参数1:已存在的文件名    参数2 要修改的文件名
```

④ 删除文件

```
remove( "d:\\bbb.txt" );//删除某个存在的文件
```

三、文本文件读写操作

文本文件：即以人看得懂的字符方式存储信息。文本文件可以用任何文本编辑类软件打开（比如：记事本， editplus等）



① 文本文件“写”操作——覆盖重写模式

```
#include<stdio.h>
#include<stdlib.h>
int main  ()
{
    FILE* writeFile = fopen("d:\\aa.txt","wt");//wt以 覆写的方式 打开文本文件
    if (writeFile)//如果文件打开成功
    {
        fprintf(writeFile,"%s\n%d","梁山名单",108);//向IO流缓冲区输出格式化字符串。还可用fputs输出纯字符串
        fclose(writeFile);//关闭文件流。
        writeFile=NULL;//释放文件指针。
    }
    else
    {
        printf("创建失败\n");
    }
    return 0;
}
```

② 文本文件“写”操作——打开追加模式

```
FILE* writeFile = fopen("d:\\bbb.txt","at");//at以追加模式 打开 文本问耳机
if (writeFile)//如果文件打开成功
{
    fputc('A',writeFile);//向IO流缓冲区写入一个字符
    fputs("西游记",writeFile);//向IO流缓冲区写入一串字符
    fclose(writeFile);//关闭文件流。
    writeFile=NULL;//释放文件指针。
}
else
{
    printf("创建失败\n");
}
```

③ 文本文件“写”操作——读写混合方式

```
FILE* rwFile = fopen("d:\\bbb.txt","r+t");//r+t 以读写混合模式打开文本文件
if (rwFile)//如果文件打开成功
{
    //确定写入的光标位置。
    //fseek(rwFile,0,SEEK_CUR);//当前光标位置 后偏移0个位置 准备写入:
    //fseek(rwFile,0,SEEK_END);//文件尾部位置 后偏移0个位置 准备写入:
    fseek(rwFile,4,SEEK_SET);//从文件开头位置偏移4个字节后
    fputs("哈利波特",rwFile);
    fclose(rwFile);//关闭文件流。
    rwFile=NULL;//释放文件指针。
}
else
{
    printf("创建失败\n");
}
```

④ 文本文件“读”操作——读取若干个整数

```
#include<stdio.h>
int main ()
{
    //1 读取若干个整数
    FILE * readFile=fopen("d:\\aa.txt","rt");//rt以只读的方式打开文件
    int arr[20]={0};
    int i ;
    int len=0;
    if(readFile)
    {
        while(!feof(readFile))//feof函数判断是否到了文件的尾部， !feof代表没有，feof--
file end of file
        {
            fscanf(readFile,"%d,",&arr[len++]);//以%d,的格式读取每一个整数到数组某地
址。
        }
        fclose(readFile);
        readFile = NULL;
```

```
    }
    //验证数组内容
    for( i=0 ; i<len;i++ )
    {
        printf("%d\n",arr[i]);
    }
    return 0;
}
```

⑤文本文件“读”操作——读取若干行字符串

```
FILE * readFile=fopen("d:\\白名单.txt","a+t");
char str[101];//字符串容器
if(readFile)
{
    while(!feof(readFile))
    {
        fgets(str,100,readFile);//从IO流读取100个字符存入str字符数组
        printf("%s",str);//打印字符串，以验证。
    }
    fclose(readFile);
}
```

⑥ rewind函数：返回文件头部的范例

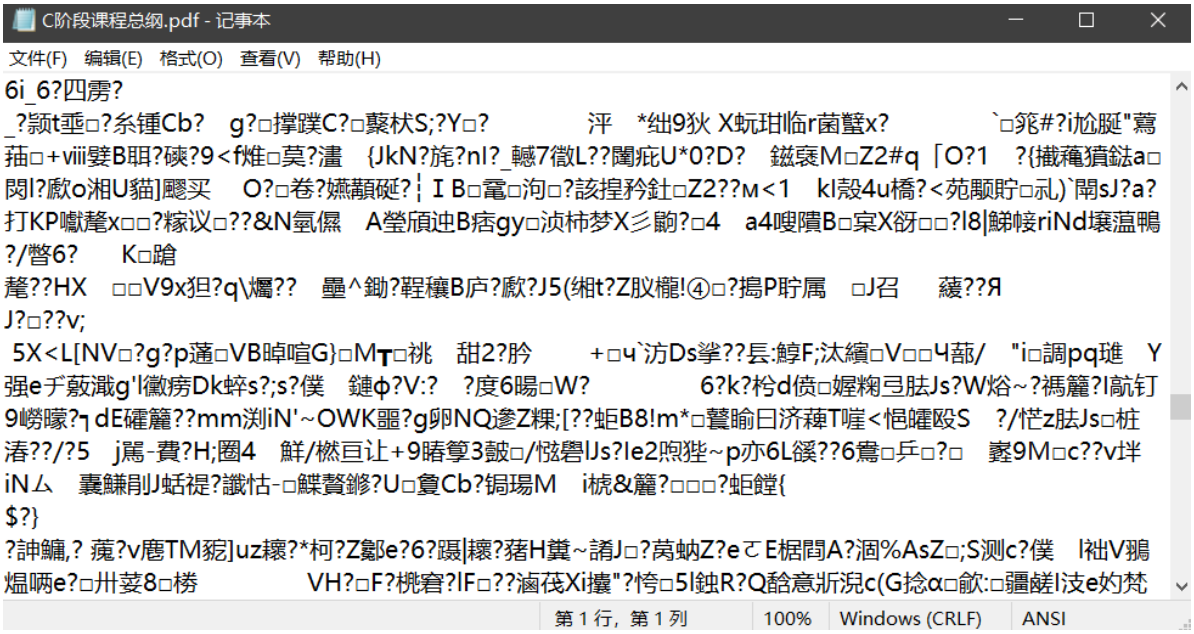
```
#include<stdio.h>
int main ()
{
    FILE * readFile=fopen("d:\\白名单.txt","a+t");
    char str[100];//字符串容器
    int i=0;
    if(readFile)
    {
        again: //设置行标记 用于goto语句的跳转
        while(!feof(readFile))
        {
            fgets(str,100,readFile);
            printf("%s",str);
        }
        if(i!=0) goto over;// 跳转到 over标记的行
        printf("\n再来一次\n");
        rewind(readFile);//返回到文件的头部
        i++;
        goto again;
        over://设置行标记 用于goto语句的跳转
        fclose(readFile);
    }
```

```
    return 0;
}
```

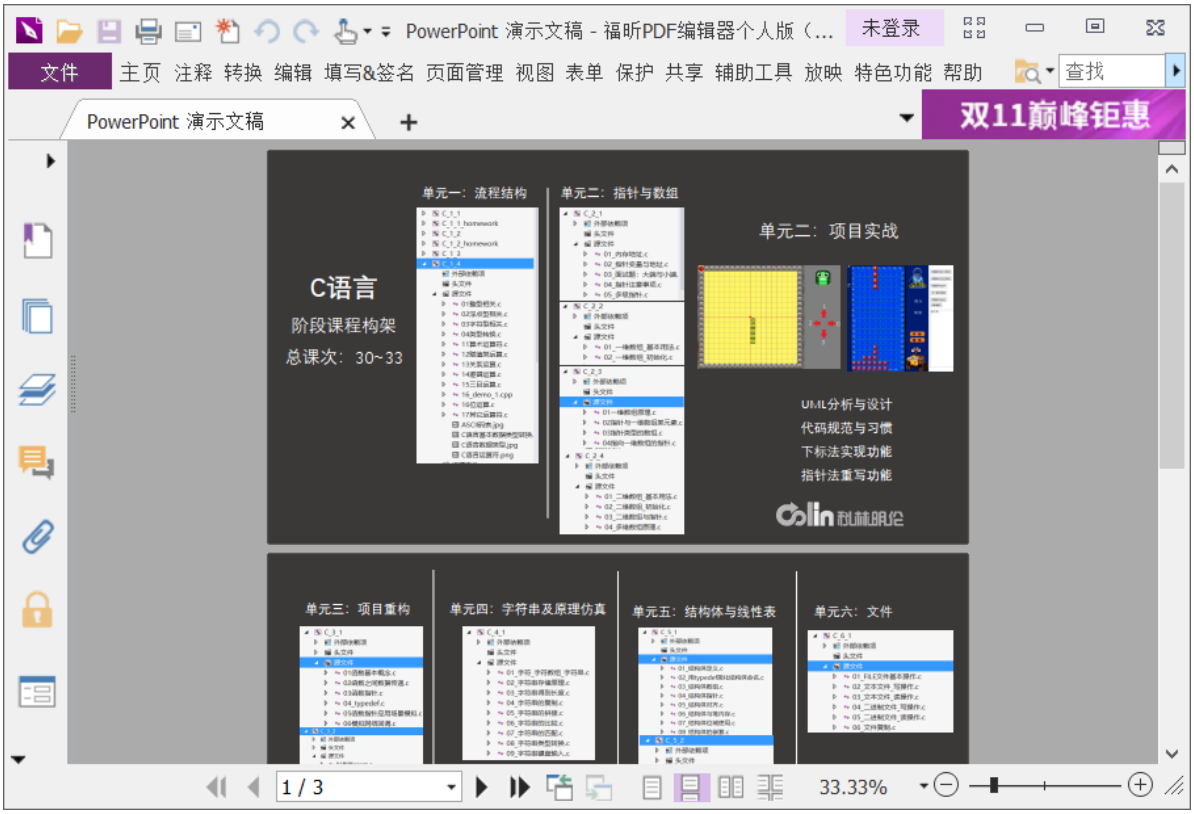
【注意：】goto语句由于其过于灵活乱跳，通常不在开发中使用。

四、二进制文件的读写操作

所谓二进制文件就是文件的内容就是内存中01数据的存储，即拿普通记事本以字符文本的方式查看是“乱码”的模样比如：如果用记事本查看：“C阶段课程总纲.pdf”，看到的是如下的样子：



用专门的工具软件才能读取识别里面信息，比如pdf阅读器，打开就是如下的样子：



所以读二进制文件的理解就是：**数据在内存中的样子，直接存入到文件，不需要转换为字符的模样，存取速度快于文本文件**

① 二进制文件“写”操作

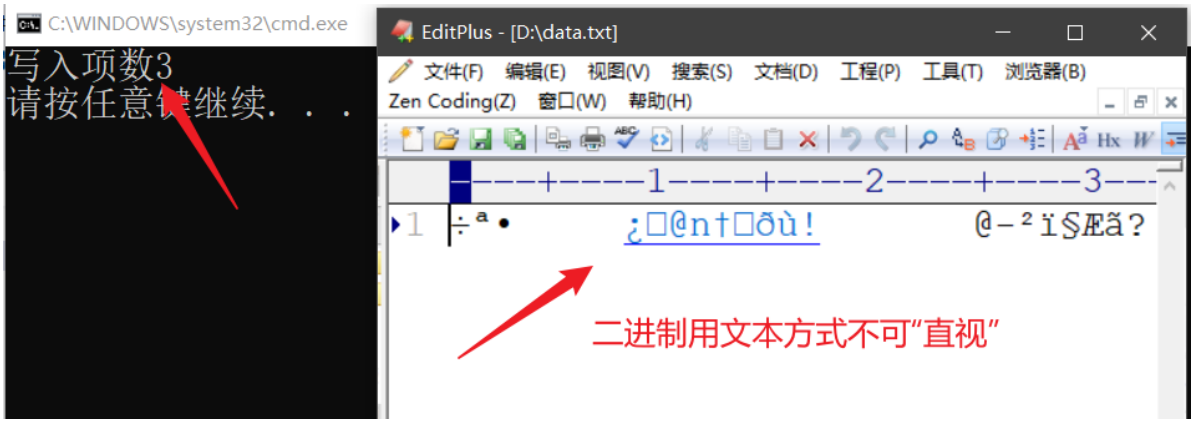
A: 向二进制文件写入一个整数

```
#include <stdio.h>
#include <string.h>
int main  ()
{
    FILE * writeFile=fopen("d:\\data.txt","wb");
    int a=100;
    //数据源的地址，一个成员大小，最大数量，文件指针
    fwrite(&a,sizeof(int),1,writeFile);
    fclose(writeFile);
    return 0;
}
```

从结果： d□□□ 可以看出不是字符形式的100而是内存中的二进制形式。不可用文本直接“读懂”

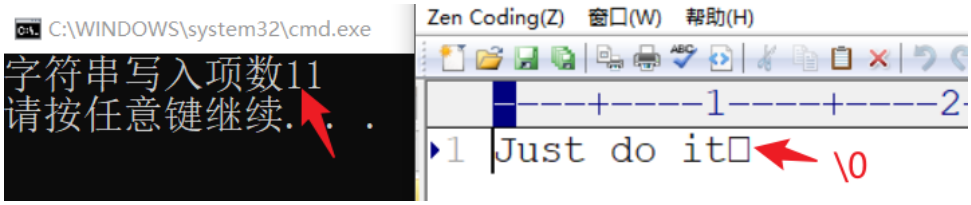
B: 向二进制文件写入一个数组

```
double arr[]={2.71828,3.14159,0.618};
//1 一次性写入数组的3个成员
int xs = fwrite(arr,sizeof(double),3,writeFile);//fwrite返回成功写入到文件的数据项数。
printf("写入项数%d\n",xs);
```



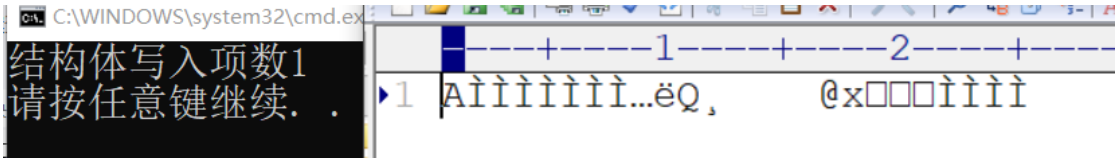
C：向二进制文件写入一个字符串

```
char*str="Just do it";
int xs=fwrite(str,sizeof(char),strlen(str)+1,writeFile );//strlen(str)+1 此处+1是把\0的位置也算进去
printf("字符串写入项数%d\n",xs);
```



D：向二进制文件写入一个结构体

```c
typedef struct
{
    char a;
    double b ;
    int c;
} AAA;
AAA aaa={'A',3.14,888};
int xs=fwrite(&aaa,sizeof(AAA), 1,writeFile );
printf("结构体写入项数%d\n",xs);
```



② 二进制文件“读”操作

A： 从二进制文件读取一个整数

```c
#include <stdio.h>
int main  ()
{
    FILE * readFile=fopen("d:\\data.txt","rb");//rb以只读模式打开二进制文件
    int a ;
    //存入地址 读取字节数 读取几个 IO流文件指针
    fread(&a,sizeof(int),1,readFile);
    printf("a=%d\n",a);
    fclose(readFile);
    return 0;
}
```

B： 从二进制文件读取一组整数

```c
double arr[3],d  ;
int i,g;
fread(arr,sizeof(arr),1,readFile);//一次读一组数到double数组。
for(i=0 ; i<3;i++ )
{
    printf("arr[%d]=%f\n",i,arr[i]);
}
rewind(readFile);//回到文件开头
for(i=0 ; i<3 ; i++  )
{
    g=fread(&d,sizeof(double),1,readFile);//一次读一个小数到double变量
    printf("读了%d个,数字为%f\n",g,d);
}
```

```
double arr[]={2.71828,3.14159,0.618};
//1 一次性写入数组的3个成员
int xs = fwrite(arr,sizeof(double),3,writeF
printf("写入项数%d\n",xs);
```

存入

```
C:\WINDOWS\system32\cmd.exe
arr[0]=2.718280
arr[1]=3.141590
arr[2]=0.618000
读了1个，数字为2.718280
读了1个，数字为3.141590
读了1个，数字为0.618000
请按任意键继续. . .
```

取出

C： 从二进制文件读取一个字符串

```
char str[100];//定义一个字符数组存储读取的数据
int xs=fread(str,sizeof(char),11,readFile );//每个读取单元1个字节，一次从IO流最多读取11个
printf("字符串读出项数%d 字符串：%s\n",xs,str);
```

```
char*str="Just do it";
int xs=fwrite(str,sizeof(char),strlen(str)+1,writeFile )
printf("字符串写入项数%d\n",xs);
```

存入

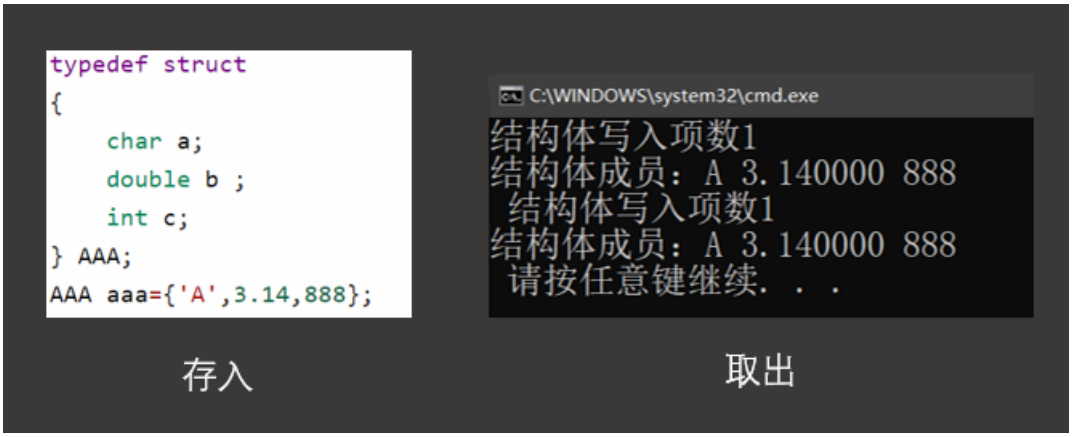
```
C:\WINDOWS\system32\cmd.exe
字符串读出项数11 字符串：Just do it
请按任意键继续. . .
```

取出

D： 从二进制文件读取一个结构体

```
typedef struct
{
    char a;
    double b ;
    int c;
} AAA;

AAA aaa ;//在栈区定义一个结构体变量
AAA * p=(AAA*)malloc(sizeof(AAA));//在堆区开辟一个能存放结构体AAA的空间
int xs=fread(&aaa,sizeof(AAA), 1,readFile );//从IO流读取一个结构体数据到aaa变量
//验证读取结果
printf("结构体写入项数%d \n",xs);
printf("结构体成员：%c %f %d\n ",aaa.a,aaa.b,aaa.c );
rewind(readFile);//回到文件开头
xs=fread(p,sizeof(AAA),1,readFile);//从IO流中读取一个结构体数据到堆空间
//验证读取结果
printf("结构体写入项数%d \n",xs);
printf("结构体成员：%c %f %d\n ",p->a,p-> b,p->c );
```



五、二进制文件的复制算法

```
#include <stdio.h>
int main()
{
    FILE * from = fopen("d:\\白名单.txt","rb");//以rb二进制只读方式打开文件
    FILE * to = fopen("d:\\bmd.txt","wb");//以wb二进制覆写的方式建立文件
    unsigned char buffer[1024];//定义1K的缓冲区。
    int len;//保存每次读取的实际字节数
    if( from&&to )//两个文件都能打开
    {
        while((len=fread(buffer,1,1024,from))>0)//每次从源文件读取1K的字节，并返回实际读取的长度 如果>0 则有数据 如果=0则数据被读取完成
        {
            fwrite(buffer,1,len,to);//把缓冲区的1K字节覆写到目标文件缓冲区
        }
        //关闭文件缓冲区
        fclose(from);
        fclose(to);
    }
    return 0;
}
```